

UC BERKELEY CS 185/285 • PRIORITY 1 COMPLETE MASTER HANDBOOK

Robotics Reinforcement Learning Foundations

An Intuitive, Beginner-Friendly Manual for Dual-Arm Soft Object Manipulation (Paper 1: Adaptive Tomato Slicing with PPO/SAC in Isaac Lab via SkRL)

Lectures Covered: CS285 L01, L04, L05, L06, L08, L10
Instructor: Prof. Sergey Levine (UC Berkeley RAIL Lab)

Research Lab: DEX-ROB Lab, Tianjin University (Prof. Shan An)

Environment: NVIDIA Isaac Lab / PhysX 5 • SkRL Framework

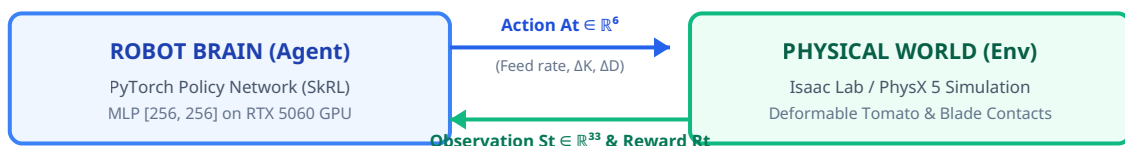
Module 1: The Foundations & The Robotics Closed Loop (Lecture 1)

(CS285 Lecture 1 • Video ID: DD8APgTEix4) Welcome to Reinforcement Learning. In computer science, we are used to writing algorithms where we know the exact rules (like sorting numbers or computing path planning via A*). In robotics, the physical world is messy: soft objects deform unpredictably, contact forces spike discontinuously, and sensors have noise.

1.1 Why Reinforcement Learning Over Supervised Learning?

In **Supervised Learning**, an expert gives the model an input x and the "correct answer" y . The model just minimizes prediction error. But in robotics:

- **No Single "Correct" Action:** There is no unique "correct" millivolt command for a robot motor. Many different knife trajectories can cut a tomato cleanly.
- **Compounding Error (Distribution Drift):** If a supervised model makes a tiny 2 mm mistake at step 10, it enters a state it has never seen before, panics, and crashes the robot (the $O(T^2)$ error problem).
- **Active Closed-Loop Control:** In RL, the robot's actions alter what it sees next. Every decision has consequences that ripple into the future.



1.2 The Credit Assignment Problem

Levine emphasizes the **Credit Assignment Problem**: If your robot cuts for 300 steps and the tomato bursts into juice at step 280, *which of the preceding 279 actions was the true cause?*

 PLAIN ENGLISH: THE EXAM ANALOGY

If you study all semester and get an **F** on the final exam, that final letter grade doesn't tell you whether you failed because you slept late on Tuesday or because you didn't understand Chapter 3. If an RL agent only receives a score at the end of an episode (+100 for cut, -50 for crush), training takes millions of episodes to isolate the mistake. This is why we design **dense progress rewards** and use **Value Functions** to evaluate state quality at every microsecond!

Module 2: The Core Mathematics & MDP Formulation (Lecture 4)

(CS285 Lecture 4 • Video ID: Fcplul7rAEE) Lecture 4 sets up the formal language of RL. Do not be intimidated by the symbols—they are simply precise mathematical shorthand for physical concepts in your laboratory.

2.1 The Markov Decision Process: $M = \langle S, A, T, R, \gamma \rangle$

Symbol	Formal Math	Your Paper 1 Physical Translation
State (s)	$s_t \in S$	33 numbers: Dual-arm joint positions & velocities (14), knife pose & velocities (9), tomato pose (7), TacBlade F/T (6), acoustic burst (1).
Action (a)	$a_t \in A$	6 numbers: Downward feed delta Δv_{des} , lateral sawing speed v_{slice} , vertical stiffness delta ΔK , damping delta ΔD , holding arm stabilization.
Dynamics (T)	$p(s_{t+1} s_t, a_t)$	Isaac Sim PhysX 5 Engine: Computes knife penetration, tomato deformation, friction, and outputs the next sensor state.
Reward (r)	$r(s_t, a_t)$	Scoring function: Penetration progress + Sawing bonus - Crushing force penalty (>8N) - Jitter penalty.
Discount (γ)	$\gamma = 0.99$	Patience factor: Ensures the policy plans for the complete cut rather than stopping on first skin contact.

2.2 The Markov Property: The "Goldfish Memory" Rule



WHY VELOCITIES AND FORCES ARE MANDATORY IN STATE SPACE

A system has the **Markov property** if $p(s_{t+1} | s_t, a_t) = p(s_{t+1} | s_t, a_t, s_{t-1}, \dots, s_1)$. The future depends **only on the present state**, not on how you got there.

If you only passed knife position z to your network, knowing $z = 5$ cm does not tell you if the blade is plunging downward at high speed or retracting upward. By including **linear/angular velocities** and **rates of force change**, step s_t contains 100% of the information needed to predict step s_{t+1} .

2.3 Value Functions: The AI "Fortune Teller"

Instead of waiting until the end of a cut to know if an action was good, RL algorithms fit **Value Functions**:

- **State-Value Function $V^\pi(s)$:** "How good is it to be in this knife position?" Expected total future score from state s onward.
- **Action-Value Function $Q^\pi(s, a)$:** "How good is taking action a in this position?" Expected future score if you pick action a right now, and then continue cutting normally.
- **Advantage Function $A^\pi(s, a) = Q(s, a) - V(s)$:** "Was this action better than average?"
 - If $A > 0$: Outstanding knife adjustment! Increase its probability.
 - If $A < 0$: Terrible knife adjustment! Suppress its probability.

Module 3: Direct Policy Optimization & Policy Gradients (Lecture 5)

(CS285 Lecture 5 • Video ID: S0D9REIVdg4) How do we actually update the neural network weights θ so the robot gets better at cutting? This is the domain of **Policy Gradients**.

3.1 The Fundamental Dilemma: You Cannot Differentiate Tomato Physics

In standard deep learning (e.g., image classification), you compute gradients by backpropagating through known mathematical layers. But in robotics:

⚠️ WHY DIRECT BACKPROPAGATION FAILS THROUGH PHYSICS

A tomato skin tearing under a blade is a **discontinuous physical fracture**. Contact friction, plastic pulp flow, and skin rupture cannot be differentiated with standard calculus. If you try to calculate $d(\text{Tomato_State}) / d(\text{Motor_Voltage})$ analytically, the derivative is either undefined or zero.

3.2 The Policy Gradient Theorem & The Log-Derivative Trick

Prof. Levine explains how RL bypasses this dilemma entirely. We take the gradient of the *expected reward* using the **log-derivative trick**:

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) \cdot \hat{Q}_{i,t}$$

💡 PLAIN ENGLISH: TRIAL AND ERROR FORMALIZED

Look at the two halves of that formula:

1. $\nabla_{\theta} \log \pi_{\theta}(\mathbf{a} | \mathbf{s})$: Which direction in weight space makes action a more probable?
2. Q (The Score / Multiplier): How good was that action?

The whole algorithm is just: If an action led to a clean cut with low crushing force, **Q is positive**, so we nudge the weights to make that action more likely. If the action caused a high crushing penalty, **Q is negative**, so we nudge the weights in the opposite direction!

3.3 Continuous Gaussian Policies for Robot Control

In games like Pong, actions are discrete (up or down). But in your dual-arm setup, the robot must output continuous real numbers (e.g., downward feed rate = 2.45 mm/s, stiffness $\Delta K = 350$ N/m).

Levine shows how to parameterize a **Continuous Gaussian Policy**: The neural network takes the 33-dimensional state $\mathbf{s}_t$ and outputs the **mean $\mu_{\theta}(\mathbf{s})$** and **standard deviation $\sigma_{\theta}(\mathbf{s})$** of a normal distribution:

$$\pi_{\theta}(\mathbf{a} | \mathbf{s}) = \frac{1}{\sqrt{(2\pi\sigma^2)}} \exp\left(-\frac{(\mathbf{a} - \mu_{\theta}(\mathbf{s}))^2}{2\sigma^2}\right)$$

When training in Isaac Lab, the robot samples its actions: $\mathbf{a}_t \sim \mathcal{N}(\mu_{\theta}(\mathbf{s}_t), \sigma_{\theta}(\mathbf{s}_t))$. The noise σ allows the robot to explore different knife motions!

3.4 Reducing Noise: Causality & Baselines

Vanilla policy gradients suffer from high variance (noise). Levine introduces two vital variance reduction techniques used in all modern code:

1. **Causality ("Reward-to-Go")**: Actions taken at step 50 cannot possibly affect rewards earned at step 10.

We only sum rewards from the current step forward: $G_t = \sum_{t'=t}^T \gamma^{t'-t} r_{t'}$.

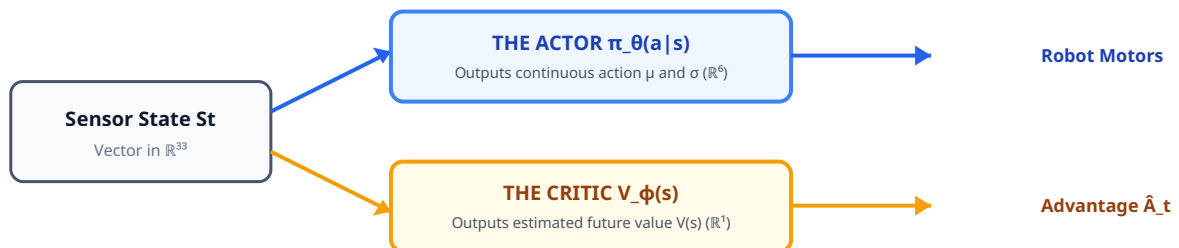
2. **State-Dependent Baseline $b(s)$:** What if all rewards are positive? Even mediocre knife actions get pushed up! By subtracting the average expected score $b(s) = V(s)$, only actions that are *better than average* get reinforced. Subtracting a baseline is **completely unbiased**!

Module 4: Actor-Critic Architectures & GAE (Lecture 6)

(CS285 Lecture 6 • Video ID: MzIWnZrCvw) Policy gradients reduce variance by using baselines. But Monte Carlo rollouts (running the knife to the end of the cut) are still noisy. **Actor-Critic** replaces noisy rollouts with a learned Critic network.

4.1 The Dual-Network Architecture

In SkRL, your policy is not a single network. It is split into two specialized sub-networks:



4.2 Temporal Difference (TD) Learning: Updating the Critic

How does the Critic learn to predict the future? It uses **Temporal Difference (TD) error**:

$$\delta_t = r(s_t, a_t) + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$$

"My prediction right now should match the reward I just got plus my prediction of what happens next." The Critic minimizes this error using standard mean-squared error (MSE) regression.

4.3 Generalized Advantage Estimation (GAE-λ)

In Slide 24, Levine introduces **GAE**. We face a trade-off:

- **1-Step TD (Critic only)**: Low variance, but high bias (if the Critic is inaccurate, the Actor learns bad actions).
- **Full Monte Carlo (Rollouts only)**: Zero bias, but massive variance (noisy).

GAE blends them using a decay parameter $\lambda \in [0, 1]$:

$$\hat{A}_t^{\text{GAE}} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}$$

⚙️ SKRL CONFIGURATION MAPPING

In your SkRL configuration file (`ppo_cfg.yaml`), you will see:

`gae_lambda: 0.95`

This 0.95 is the GAE parameter! It gives you 95% of the variance-reduction benefits of the Critic while relying on multi-step rollouts to eliminate bias.

Module 5: Continuous Control & Soft Actor-Critic (Lecture 8)

(CS285 Lecture 8 • Video ID: IQaVa53pS-Q) In this lecture, Levine explains why classic Q-learning (like DQN that beat Atari) completely fails on robotic arms, and derives **Soft Actor-Critic (SAC)**.

5.1 The Continuous Action Trap: Why DQN Breaks on Robots

In Q-learning, the optimal action is chosen via: $\mathbf{a}^* = \arg \max_{\mathbf{a}} Q(\mathbf{s}, \mathbf{a})$.

- **In Atari Games:** There are only 4 buttons (Up, Down, Left, Right). You calculate $Q(\mathbf{s}, \mathbf{a})$ for all 4, and pick the highest. Easy!
- **On a Robot Arm:** Actions are continuous 6-dimensional vectors of real numbers (e.g., torques $\in [-10.0, +10.0]$). Finding the maximum across infinite continuous values at 1,000 Hz is impossible!

The Fix: Instead of searching for the max action by brute force, train an **Actor network** to directly output the action that maximizes the Critic's Q-value!

5.2 Soft Actor-Critic (SAC) & Maximum Entropy Exploration

In traditional RL, the robot only cares about reward points. In **Maximum Entropy RL**, the objective adds an **entropy bonus**:

$$J(\pi) = \sum_t E [r(\mathbf{s}_t, \mathbf{a}_t) + \alpha H(\pi(\cdot | \mathbf{s}_t))]$$

💡 PLAIN ENGLISH: PAYING THE ROBOT TO BE CURIOUS

Entropy $H(\pi)$ measures how random/broad an action distribution is:

- High entropy: The robot tries many different sawing speeds and blade angles.
- Low entropy: The robot always does the exact same motion deterministically.

Why this is crucial for tomato cutting: If a robot only maximizes reward, it quickly gets terrified of crushing the tomato and freezes just above the skin (a local minimum). The entropy temperature α pays the robot to keep experimenting and trying diverse contact dynamics!

5.3 Practical Stabilization Tricks in Modern Q-Learning

Levine covers three mandatory stabilization mechanisms used in SAC and SkRL:

Mechanism	How It Works	Why It Stops Training Collapse
Target Networks (Q)	A separate copy of the Critic weights updated slowly: $\bar{Q} \leftarrow \tau Q + (1-\tau) \bar{Q}$.	Stops the network from chasing a moving target (prevents gradient explosion).
Twin Q-Networks (Double Q)	Train two independent Critics Q_1 and Q_2 , and use $\min(Q_1, Q_2)$.	Eliminates <i>positive maximization bias</i> (prevents the robot from over-optimistically believing a dangerous knife slam is good).
Replay Buffer	Stores 1,000,000 past transitions ($\mathbf{s}, \mathbf{a}, r, \mathbf{s}'$) and samples random mini-batches.	Breaks temporal correlation between consecutive simulator frames.

Module 6: Advanced Policy Gradients & PPO (Lecture 10)

(CS285 Lecture 10 • Video ID: m7IU5KBS4sw) Proximal Policy Optimization (PPO) is the undisputed gold standard of modern robot learning. This lecture explains why PPO was invented and how its clipping mechanism prevents policy destruction.

6.1 The Nightmare of "Policy Collapse"

In supervised learning, if you take a step that is too large, the error increases slightly on that epoch, but you easily recover on the next batch.

☀ THE CATASTROPHIC FAILURE MODE OF RL: POLICY COLLAPSE

In RL, **the policy collects its own data**. If an aggressive gradient update moves the policy weights into a bad region, the robot starts flailing wildly. All 1,024 parallel environments in Isaac Sim generate pure garbage trajectories. Because the new data is garbage, the next gradient update is even worse. The policy enters an unrecoverable death spiral!

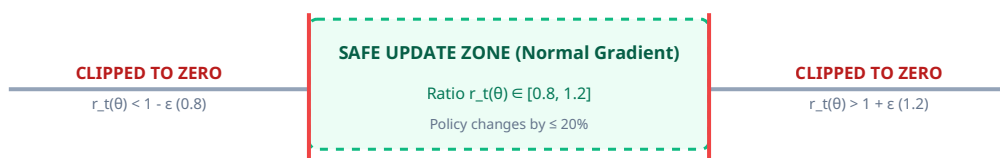
6.2 The Solution: The PPO Clipped Surrogate Objective

PPO solves policy collapse by introducing a "speed limiter" on policy updates. It defines the **importance sampling ratio**:

$$r_t(\theta) = \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t) / \pi_{\theta_{\text{old}}}(\mathbf{a}_t | \mathbf{s}_t)$$

This ratio measures how much the new policy deviates from the old policy that collected the data. PPO's loss function is:

$$L^{\text{CLIP}}(\theta) = \hat{E}_t [\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t)]$$



💡 WHY PPO IS THE UNDISPUTED KING OF ISAAC LAB

In vanilla policy gradients, you can only take **one single gradient step** on your data before throwing it away. With PPO, because the clipping mechanism guarantees safety, you can train for **4 to 8 epochs** on the exact same batch of parallel Isaac Lab rollouts! This dramatically speeds up GPU training.

Module 7: The Master Paper 1 Implementation Blueprint

Here is the complete operational synthesis connecting CS285 Priority 1 directly to your Master's thesis methodology and code in **Isaac Lab + SkRL**:

7.1 The Complete State & Action Space Mapping

Variable	Dimension	Physical Meaning & Sensor Origin
$\mathbf{q}_{\text{dual}}, \dot{\mathbf{q}}_{\text{dual}}$	$\mathbb{R}^{14}$	Joint angles and angular velocities for both 7-DoF arms (7 cutting arm + 7 holding arm).
$\mathbf{p}_{\text{knife}}, \mathbf{v}_{\text{knife}}, \boldsymbol{\omega}_{\text{knife}}$	$\mathbb{R}^9$	3D position, linear velocity, and angular velocity of the sensorized blade tip.
$\mathbf{p}_{\text{tomato}}, \mathbf{R}_{\text{tomato}}$	$\mathbb{R}^7$	3D position and orientation quaternion of the target tomato (tracked via YOLO/RGB-D in real, direct USD in sim).
$\mathbf{F}_{\text{TacBlade}}$	$\mathbb{R}^6$	6-axis force/torque vector measured at the blade root (normal force F_z , lateral friction F_x).
E_{burst}	$\mathbb{R}^1$	Acoustic burst energy envelope (100 Hz - 5 kHz) detecting the exact millisecond of skin puncture.
Total State $\mathbf{S}$	$\mathbb{R}^{33}$	Full Markovian observation vector fed into SkRL Actor & Critic networks.

7.2 The Reward Function Code Blueprint

In your Isaac Lab environment (`tomato_cutting_env.py`), the reward function translates Lecture 4's cumulative objective into simple tensor arithmetic:

 PYTORCH REWARD FUNCTION IMPLEMENTATION

```
def compute_rewards(self):
    # 1. Penetration progress reward (downward progress through tomato height)
    z_progress = torch.clamp(self.prev_knife_z - self.knife_z, min=0.0)
    r_pen = 5.0 * z_progress

    # 2. Lateral sawing reward (shearing motion reduces required normal force)
    v_slice = torch.abs(self.knife_vel[:, 0])
    in_contact = (self.blade_force[:, 2] > 0.5).float()
    r_slice = 2.0 * v_slice * in_contact

    # 3. Puncture & crushing force penalty (penalize normal force exceeding 8N skin threshold)
    excess_force = torch.clamp(self.blade_force[:, 2] - 8.0, min=0.0)
    p_crush = -3.0 * torch.square(excess_force)

    # 4. Action jitter penalty (ensures smooth motor compliance commands)
    action_diff = self.actions - self.prev_actions
    p_smooth = -0.05 * torch.sum(torch.square(action_diff), dim=-1)

    # 5. Terminal success / failure bonuses
    r_terminal = torch.zeros_like(r_pen)
    r_terminal = torch.where(self.reached_board & (excess_force == 0), r_terminal + 100.0, r_terminal)
    r_terminal = torch.where(self.blade_force[:, 2] > 25.0, r_terminal - 50.0, r_terminal)

    return r_pen + r_slice + p_crush + p_smooth + r_terminal
```

7.3 Master SkRL Configuration ('ppo_cfg.yaml')

Every single hyperparameter in SkRL now has a direct theoretical justification from CS285:

 PPO_CFG.YAML REFERENCE

```
algorithm:
  class: PPO
  clip_range: 0.2          # Lecture 10: Prevents policy collapse by limiting ratio to [0.8, 1.2]
  gae_lambda: 0.95        # Lecture 6: GAE variance-bias trade-off parameter
  discount_factor: 0.99    # Lecture 4:  $\gamma = 0.99$  ensures the robot cuts all the way to the board
  learning_rate: 3e-4      # Standard Adam step size for deep neural policy networks
  learning_rate_scheduler: KLRAdaptiveLR # Automatically lowers LR if policy changes too fast
  entropy_loss_scale: 0.01 # Lecture 8: Encourages stochastic exploration of sawing angles
  value_loss_scale: 1.0    # Critic MSE loss weight
  epochs: 5               # Lecture 10: PPO safely runs 5 epochs per parallel simulation batch
  mini_batches: 4         # Mini-batch gradient descent subdivisions
```

Module 8: Thesis Defense Master Cheatsheet (Top 10 Q&A)

When defending your methodology before Prof. Shan An or IEEE reviewers, be prepared for these 10 core theoretical questions:

#	Reviewer / Advisor Question	Your Bulletproof Academic Answer
1	Why use Reinforcement Learning instead of standard Impedance Control?	Classical impedance control requires hand-tuning stiffness (K) and damping (D) for uniform materials. Soft fruits have non-linear fracture transitions where mechanical stiffness drops 80% in milliseconds. RL dynamically modulates compliance parameters in response to high-frequency tactile bursts.
2	Why is your task formulated as a Markov Decision Process?	Because our 33-dimensional state vector contains both kinematics (positions and velocities) and contact forces, meaning the transition to the next state depends solely on the current state and action, strictly satisfying the Markov property.
3	Why did you choose PPO over Model-Based RL?	Deformable viscoplastic fracture mechanics are notoriously difficult to model analytically. Model-based RL compounds model errors, causing real-world transfer failure. PPO is model-free, robust, and directly optimizes the true reward objective.
4	How do you address PPO's low sample efficiency?	Through GPU-vectorized parallel simulation in NVIDIA Isaac Lab. By running 1,024 environments simultaneously on our RTX GPU, we collect over 10 million simulation transitions in under 45 minutes, rendering sample efficiency irrelevant.
5	What is the role of the Critic network?	The Critic approximates the State-Value function $V(s)$ using Temporal Difference learning. It is used to compute the Advantage function $A(s, a) = Q(s, a) - V(s)$, which dramatically reduces policy gradient variance compared to raw Monte Carlo rollouts.
6	Why is Generalized Advantage Estimation (GAE) with $\lambda = 0.95$ used?	Pure TD learning has low variance but introduces bias if the Critic is imperfect. Pure rollouts have zero bias but high variance. GAE with $\lambda = 0.95$ achieves an optimal exponential blend of multi-step returns.
7	Why did you configure continuous action spaces instead of discrete?	Robot joint motor torques, feed velocities, and compliance deltas (ΔK , ΔD) are continuous physical quantities. Discretizing them creates coarse, jerky motions that damage soft object skins.
8	What is the purpose of PPO's clipping mechanism?	It bounds the probability ratio $r_t(\theta)$ between $[1-\epsilon, 1+\epsilon]$ (typically $\epsilon = 0.2$). This prevents policy collapse by discarding gradient steps that deviate too far from the policy that generated the trajectory data.
9	Why is lateral sawing velocity rewarded?	Under fracture mechanics shear-stress coupling, lateral slicing motion significantly reduces the required normal downward force (F_z) to achieve fracture toughness (K_{Ic}), preventing fruit squishing.
10	How does Soft Actor-Critic (SAC) prevent local minima?	By optimizing Maximum Entropy RL (Reward + α -Entropy). The entropy term encourages broad, stochastic exploration, preventing the robot from adopting timid

#	Reviewer / Advisor Question	Your Bulletproof Academic Answer
		policies that freeze above the tomato skin to avoid crushing penalties.

DEX-ROB Lab (Tianjin University) Master's Research Program • Prof. Shan An • Created with Antigravity AI